

PROJETO: GASTROHUB

Sistema de Gestão para Restaurantes

Relatório Técnico: Fase 2

FIAP — PÓS TECH · ARQUITETURA E DESENVOLVIMENTO JAVA

Aluno: Geovanne de Souza Paes

RM: 372009

Aluno: Rafael Teles Espindola

RM: 370772

Aluno: Victor Silva Mazzola

RM: 370764

Aluno: Gideão Lucas de Souza Silva

RM: 374230

Aluno: Luis Filipi Santos Souza

RM: 372233



Link do YouTube para vídeo de apresentação do projeto:

<https://youtu.be/wW2IT7NuN4s?si=7yzHaEoUEMV7--xw>

Sumário

1. Introdução
2. Arquitetura do sistema
3. Modelagem das entidades e Banco de dados
4. Descrição dos endpoints da API
5. Tratamento de erros
6. Documentação Swagger / OpenAPI
7. Configuração do Docker
8. Qualidade do código
9. Testes automatizados
10. Collection Postman
11. Repositório do código
12. Guia passo a passo para testar o projeto
13. Observações e Limitações conhecidas

1. Introdução

Descrição do Problema

Na região, um grupo de restaurantes decidiu contratar estudantes para construir um sistema de gestão compartilhado para seus estabelecimentos. A decisão foi motivada pelo alto custo de sistemas individuais, levando os restaurantes a se unirem em torno de uma solução única. Esse sistema permitirá que os clientes escolham restaurantes com base na comida oferecida, em vez de se basearem na qualidade do sistema de gestão.

Objetivo do projeto

O objetivo do projeto é construir o backend do sistema de gestão compartilhado em **Spring Boot**, contemplando a gestão de tipos de usuários, o cadastro de usuários, o cadastro de restaurantes e o cadastro dos itens do cardápio. Além das funcionalidades, o projeto prioriza práticas de desenvolvimento e estruturação de código limpo: foi organizado em **Clean Architecture**, com separação clara entre domínio, casos de uso, infraestrutura e apresentação, suporte a documentação via Swagger e testes automatizados.

Nesta fase, o sistema cobre:

- CRUD de **tipos de usuário** (`UserType`), com categorias base `CLIENT` e `OWNER`
- Associação de usuários a um tipo de usuário
- CRUD de **usuários**, com garantia de unicidade de e-mail
- CRUD de **restaurantes** (nome, endereço, tipo de cozinha, horário de funcionamento e dono)
- Validação de que o dono do restaurante é um usuário existente e não é da categoria `CLIENT`
- CRUD de **itens do cardápio** (nome, descrição, preço, disponibilidade apenas no local e caminho da foto)
- Validação de que o item do cardápio pertence a um restaurante existente
- Estruturação do código em **Clean Architecture**
- Documentação via Swagger/OpenAPI
- Testes automatizados com JUnit 5, Mockito e MockMvc

2. Arquitetura do sistema

Descrição da Arquitetura

O projeto segue os princípios da **Clean Architecture**, organizado em quatro camadas: **domain**, **application**, **infrastructure** e **presentation**. As regras de negócio ficam isoladas no domínio e nos casos de uso, que dependem apenas de contratos (interfaces de *gateway*) — nunca de detalhes de banco de dados ou framework. A comunicação com o banco é feita via **Spring Data JPA / Hibernate** na camada de infraestrutura, que implementa esses contratos.

Estrutura de Pacotes

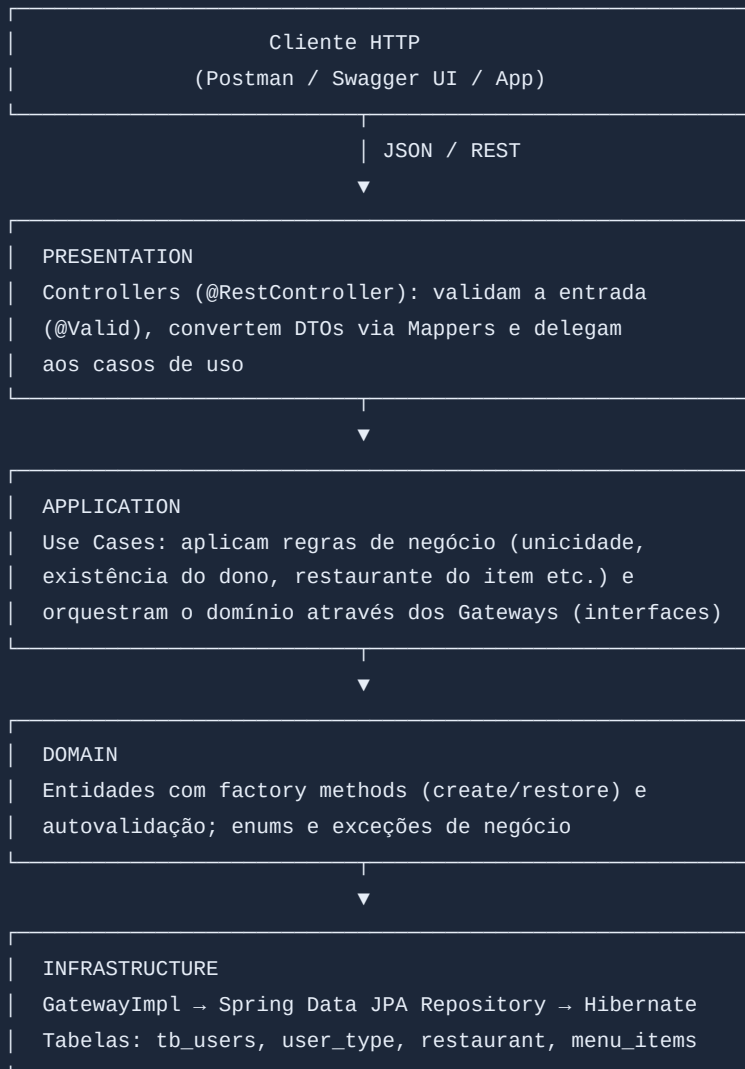
```
br.com.fiap.gastrohubapi
├── domain/                └─ Núcleo do negócio (sem dependência de framework)
│   ├── entity/           └─ User, UserType, Restaurant, MenuItem
│   ├── enums/            └─ Enumerações: BaseCategory, KitchenType
│   └── exception/        └─ Exceções de negócio (ex.: UserTypeInUseException)
├── application/          └─ Regras de aplicação
│   ├── gateway/          └─ Contratos de persistência (interfaces)
│   └── usecase/          └─ user, usertype, restaurant, menuitem
├── infrastructure/       └─ Detalhes técnicos
│   ├── config/           └─ Registro dos use cases como beans Spring
│   └── persistence/
│       ├── entity/       └─ Entidades JPA mapeadas para o banco
│       ├── repository/   └─ Repositórios Spring Data JPA
│       └── gateway/      └─ Implementações dos gateways (adapters)
└── presentation/        └─ Camada HTTP
    ├── controller/      └─ REST Controllers com documentação Swagger
    ├── dto/              └─ Request e Response records
    ├── mapper/           └─ Conversão domínio ⇌ DTO
    └── exceptionhandler/ └─ Handlers globais de erros
```

Fluxo de Dependências

```
Presentation → Application → Domain
Infrastructure → Application (implementa os gateways)
```

A camada de aplicação depende de interfaces de gateway, e não diretamente de implementações de banco. A infraestrutura implementa esses contratos usando JPA, permitindo trocar o mecanismo de persistência sem alterar as regras de negócio.

Diagrama da Arquitetura



Tecnologias Utilizadas

Camada / Função	Tecnologia
Linguagem	Java 21
Framework principal	Spring Boot 3.5.15
API REST	Spring Web MVC
Banco de dados (produção)	MySQL 8.4 (container Docker)
Banco de dados (desenvolvimento local e testes)	H2 in-memory
Driver de banco	MySQL Connector/J
ORM	Spring Data JPA / Hibernate
Validação	Bean Validation (Jakarta)
Documentação	SpringDoc OpenAPI 2.8.16 (Swagger UI)
Containerização	Docker + Docker Compose
Build	Maven (com Maven Wrapper)
Testes	JUnit 5 + Mockito + MockMvc
Utilitários	Lombok, Spring Boot DevTools

3. Modelagem das entidades e Banco de dados

Diagrama de Classes — Entidades de Domínio

O sistema possui quatro entidades de domínio: `User`, `UserType`, `Restaurant` e `MenuItem`. As entidades são criadas por meio de *factory methods* (`create` / `restore`) que executam as validações de negócio, garantindo que nenhuma instância exista em estado inválido.



Estrutura do Banco de Dados (MER)

O mapeamento objeto-relacional é feito por entidades JPA dedicadas (`UserJpaEntity` , `UserTypeJpaEntity` , `RestaurantJpaEntity` , `MenuItemJpaEntity`), mantendo o domínio livre de anotações de persistência. O schema é gerado pelo Hibernate (`ddl-auto=update`).

Tabela: user_type

Coluna	Tipo	Restrições / Descrição
id	UUID	PK, gerado pelo JPA (GenerationType.UUID)
name	VARCHAR	NOT NULL, UNIQUE: nome do tipo de usuário
base_category	VARCHAR	NOT NULL: CLIENT ou OWNER (enum como string)

Tabela: tb_users

Coluna	Tipo	Restrições / Descrição
id	UUID	PK — gerado pelo domínio (não pelo banco)
name	VARCHAR	NOT NULL: nome completo do usuário
email	VARCHAR	NOT NULL, UNIQUE: endereço de e-mail
password	VARCHAR	NOT NULL: senha do usuário
user_type_id	UUID	NOT NULL, FK → user_type.id (@ManyToOne LAZY)

Tabela: restaurant

Coluna	Tipo	Restrições / Descrição
id	UUID	PK, gerado pelo JPA (GenerationType.UUID)
name	VARCHAR	NOT NULL: nome do restaurante
address	VARCHAR	NOT NULL: endereço
kitchen_type	VARCHAR	NOT NULL: tipo de cozinha (enum como string)
opening_hours	VARCHAR	NOT NULL: horário de funcionamento
restaurant_owner_id	UUID	NOT NULL: referência ao usuário dono (validada no caso de uso)

Tabela: menu_items

Coluna	Tipo	Restrições / Descrição
id	UUID	PK, gerado pelo JPA (GenerationType.UUID)
name	VARCHAR	NOT NULL: nome do item
description	TEXT	NOT NULL: descrição do prato
price	DECIMAL(10,2)	NOT NULL: preço (maior que zero)
only_in_restaurant	BOOLEAN	NOT NULL: disponível apenas para consumo no local
photo_path	VARCHAR	Caminho de armazenamento da foto do prato (opcional)
restaurant_id	UUID	NOT NULL: referência ao restaurante (validada no caso de uso)

Enums do domínio

```
public enum BaseCategory {
    CLIENT,
    OWNER
}

public enum KitchenType {
    ITALIAN,
    JAPANESE,
    BRAZILIAN,
    MEXICAN,
    INDIAN
}
```

Associação usuário ⇌ tipo de usuário: conforme solicitado no desafio, o usuário é associado ao tipo de usuário via chave estrangeira `user_type_id`. Um tipo de usuário em uso não pode ser excluído (regra `UserTypeInUseException`).

Banco de dados definitivo (produção)

O banco de dados definitivo do projeto é o **MySQL 8.4**, executado em **container Docker** definido no `docker-compose.yml` na raiz do repositório (serviço `mysql`, imagem oficial `mysql:8.4`):

```
services:
  mysql:
    image: mysql:8.4
    container_name: gastrohub-mysql
    environment:
      MYSQL_DATABASE: gastrohub
      MYSQL_USER: gastrohub
      MYSQL_PASSWORD: gastrohub
      MYSQL_ROOT_PASSWORD: root
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
      - ./sql:/docker-entrypoint-initdb.d:ro,z
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-proot"]
```

Características da configuração:

- **Banco:** `gastrohub`, exposto na porta `3306`, com usuário dedicado `gastrohub`.
- **Persistência:** os dados são gravados no volume Docker `mysql_data`, preservados entre reinicializações do container.
- **Carga automática:** os scripts `sql/01-schema.sql` (DDL das tabelas) e `sql/02-seed.sql` (dados de exemplo) são executados automaticamente pelo container na primeira inicialização, via `/docker-entrypoint-initdb.d`.

- **Healthcheck:** o serviço da API só sobe após o MySQL responder ao `mysqladmin ping` (condição `service_healthy`).

No desenvolvimento local e nos testes automatizados, a aplicação utiliza **H2 in-memory**, dispensando qualquer instalação de banco. O driver **MySQL Connector/J** está declarado no `pom.xml` . A configuração completa do Docker é detalhada na seção 7.

4. Descrição dos endpoints da API

Base URL: `http://localhost:8080` . A API expõe quatro módulos: **Tipos de Usuário**, **Usuários**, **Restaurantes** e **Itens do Cardápio**, totalizando 24 endpoints.

Tipos de Usuário — `/user-types`

Endpoint	Método	Descrição
<code>/user-types</code>	POST	Cria um tipo de usuário
<code>/user-types</code>	GET	Lista todos os tipos de usuário
<code>/user-types/{id}</code>	GET	Busca um tipo de usuário pelo ID
<code>/user-types/{id}</code>	PUT	Atualiza um tipo de usuário
<code>/user-types/{id}</code>	DELETE	Exclui um tipo de usuário (se não estiver em uso)

Usuários — /users

Endpoint	Método	Descrição
/users	POST	Cria um usuário (e-mail deve ser único)
/users	GET	Lista todos os usuários
/users/{id}	GET	Busca um usuário pelo ID
/users/search?name=X	GET	Busca usuários pelo nome
/users/email/{email}	GET	Busca um usuário pelo e-mail
/users/{id}	PUT	Atualiza os dados de um usuário
/users/{id}	DELETE	Exclui um usuário pelo ID

Restaurantes — /restaurants

Endpoint	Método	Descrição
/restaurants	POST	Cria um restaurante (dono deve existir e não ser CLIENT)
/restaurants	GET	Lista todos os restaurantes
/restaurants/{id}	GET	Busca um restaurante pelo ID
/restaurants/search?name=X	GET	Busca restaurantes pelo nome
/restaurants/{id}	PUT	Atualiza um restaurante
/restaurants/{id}	DELETE	Exclui um restaurante pelo ID

Endpoint	Método	Descrição
/menu-items	POST	Cria um item do cardápio (restaurante deve existir)
/menu-items	GET	Lista todos os itens do cardápio
/menu-items/{id}	GET	Busca um item pelo ID
/menu-items/restaurant/{restaurantId}	GET	Lista os itens de um restaurante
/menu-items/{id}	PUT	Atualiza um item do cardápio
/menu-items/{id}	DELETE	Exclui um item pelo ID

Exemplos de Requisição e Resposta

POST /user-types: Criar Tipo de Usuário

Requisição

```
{
  "name": "Cliente",
  "baseCategory": "CLIENT"
}
```

Resposta 201: Criado com sucesso

```
{
  "id": "8d4f0a2e-6b1c-4c9a-9e2f-5a7b3c1d0e9f",
  "name": "Cliente",
  "baseCategory": "CLIENT"
}
```

Resposta 409: Nome de tipo já cadastrado

```
{
  "timestamp": "2026-07-02T14:00:00Z",
  "status": 409,
  "error": "Conflict",
  "message": "User type name already exists."
}
```


POST /users: Criar Usuário

Requisição

```
{
  "name": "Roberto Rodriguez",
  "email": "roberto.rodriguez@email.com",
  "password": "password123",
  "userId": "8d4f0a2e-6b1c-4c9a-9e2f-5a7b3c1d0e9f"
}
```

Resposta 201: Criado com sucesso (senha nunca é retornada)

```
{
  "id": "3f2b9c1e-8a4d-4e2f-9b6a-1c5d7e8f9a0b",
  "name": "Roberto Rodriguez",
  "email": "roberto.rodriguez@email.com",
  "userType": {
    "id": "8d4f0a2e-6b1c-4c9a-9e2f-5a7b3c1d0e9f",
    "name": "Cliente",
    "baseCategory": "CLIENT"
  }
}
```

POST /restaurants: Criar Restaurante

Requisição

```
{
  "name": "Cantina da Nonna",
  "address": "Av. Paulista, 1000 - São Paulo/SP",
  "kitchenType": "ITALIAN",
  "openingHours": "Seg a Dom, das 11h às 23h",
  "restaurantOwnerId": "7a1c2d3e-4f5a-6b7c-8d9e-0f1a2b3c4d5e"
}
```

Resposta 201: Criado com sucesso

```
{
  "id": "9b8a7c6d-5e4f-3a2b-1c0d-9e8f7a6b5c4d",
  "name": "Cantina da Nonna",
  "address": "Av. Paulista, 1000 - São Paulo/SP",
  "kitchenType": "ITALIAN",
  "openingHours": "Seg a Dom, das 11h às 23h",
  "restaurantOwnerId": "7a1c2d3e-4f5a-6b7c-8d9e-0f1a2b3c4d5e"
}
```

Resposta 400: Dono com categoria CLIENT

```
{
  "timestamp": "2026-07-02T14:05:00Z",
  "status": 400,
  "error": "Bad Request",
  "message": "The restaurantOwnerId cannot be a CLIENT BaseCategory User"
}
```

Resposta 409: Restaurante duplicado (mesmo nome, dono, endereço e cozinha)

```
{
  "timestamp": "2026-07-02T14:06:00Z",
  "status": 409,
  "error": "Conflict",
  "message": "Another restaurant already exists with the same name, owner, address and kitchenType"
}
```

POST /menu-items: Criar Item do Cardápio

Requisição

```
{
  "name": "Lasanha à Bolonhesa",
  "description": "Massa fresca com molho bolonhesa e queijo gratinado",
  "price": 59.90,
  "onlyInRestaurant": false,
  "photoPath": "/images/pratos/lasanha-bolonhesa.png",
  "restaurantId": "9b8a7c6d-5e4f-3a2b-1c0d-9e8f7a6b5c4d"
}
```

Resposta 201: Criado com sucesso

```
{
  "id": "5c4d3e2f-1a0b-9c8d-7e6f-5a4b3c2d1e0f",
  "name": "Lasanha à Bolonhesa",
  "description": "Massa fresca com molho bolonhesa e queijo gratinado",
  "price": 59.90,
  "onlyInRestaurant": false,
  "photoPath": "/images/pratos/lasanha-bolonhesa.png",
  "restaurantId": "9b8a7c6d-5e4f-3a2b-1c0d-9e8f7a6b5c4d"
}
```

Resposta 404: Restaurante inexistente

```
{
  "timestamp": "2026-07-02T14:10:00Z",
  "status": 404,
  "error": "Not Found",
  "message": "Restaurant not found with id: 00000000-0000-0000-0000-000000000000"
}
```

DELETE**/user-types/{id}: Excluir Tipo de Usuário**

Retorna **204 No Content** em caso de sucesso (sem corpo de resposta).

Resposta 409: Tipo de usuário em uso

```
{
  "timestamp": "2026-07-02T14:12:00Z",
  "status": 409,
  "error": "Conflict",
  "message": "Cannot delete user type because it is in use."
}
```

5. Tratamento de erros

O tratamento de erros é centralizado em dois handlers globais (`@RestControllerAdvice`):

`GlobalExceptionHandler` (usuários, restaurantes e itens do cardápio) e `UserTypeExceptionHandler` (tipos de usuário). As exceções de negócio são lançadas pelo domínio e pelos casos de uso, e convertidas em respostas HTTP com corpo padronizado.

Formato padrão

```
{
  "timestamp": "2026-07-02T14:00:00Z",
  "status": 404,
  "error": "Not Found",
  "message": "Descrição detalhada do problema"
}
```

Erros de Validação (400)

Para falhas de Bean Validation (`@Valid`), o primeiro erro de campo é retornado no formato `campo: mensagem` :

```
{
  "timestamp": "2026-07-02T14:00:00Z",
  "status": 400,
  "error": "Bad Request",
  "message": "name: Name is required."
}
```

Mapeamento de Exceções

Exceção	HTTP / Situação
UserNotFoundException	404: Usuário não encontrado
UserTypeNotFoundException	404: Tipo de usuário não encontrado
RestaurantNotFoundByIdException	404: Restaurante não encontrado
MenuItemNotFoundException	404: Item do cardápio não encontrado
UserAlreadyExistsException	409: E-mail de usuário já cadastrado
DuplicateUserNameException	409: Nome de tipo de usuário já cadastrado
UserTypeInUseException	409: Tipo de usuário em uso não pode ser excluído
RestaurantAlreadyExistsException	409: Restaurante duplicado
UserTypeNotAllowedForRestaurantOwnerException	400: Dono do restaurante não pode ser CLIENT
InvalidUserTypeException	400: Dados inválidos do tipo de usuário
IllegalArgumentException (validações do domínio)	400: Dados inválidos
MethodArgumentNotValidException	400: Erro de validação (Bean Validation)

6. Documentação Swagger / OpenAPI

A documentação interativa é gerada automaticamente pelo **SpringDoc OpenAPI 2.8.16**. Com a aplicação rodando, acesse:

- **Swagger UI:** `http://localhost:8080/swagger-ui.html`
- **Spec JSON:** `http://localhost:8080/v3/api-docs`

Estratégia de Documentação

Cada controller é anotado com `@Tag` para agrupamento e cada operação com `@Operation` e `@ApiResponse`, descrevendo os códigos de resposta possíveis (200, 201, 204, 400, 404, 409).

Endpoints documentados no Swagger

O Swagger documenta os 24 endpoints da API, agrupados em quatro tags:

Tag	Descrição
User Types	User type management endpoints
Users	User management endpoints
Restaurants	Restaurant management endpoints
Menu Items	CRUD for restaurant menu items

O Swagger UI permite executar qualquer endpoint diretamente pelo navegador com o botão **"Try it out"**, exibindo os modelos de requisição/resposta e os códigos de status.

7. Configuração do Docker

A containerização é composta por um `Dockerfile` (imagem da API) e um `docker-compose.yml` (orquestração da API com o banco MySQL), ambos na raiz do repositório.

Dockerfile (multi-stage build)

A imagem da API é construída em dois estágios, gerando uma imagem final enxuta, apenas com o JRE e o JAR da aplicação:

```
# Build stage
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -B dependency:go-offline
COPY src ./src
RUN mvn -B clean package -DskipTests

# Run stage
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

- **Estágio de build:** usa Maven + JDK 21 para compilar e empacotar o projeto (o cache de dependências é aproveitado ao copiar o `pom.xml` antes do código).
- **Estágio de execução:** usa apenas o JRE 21 (`eclipse-temurin:21-jre`), reduzindo o tamanho da imagem final.

docker-compose.yml

O Compose define dois serviços:

Serviço	Descrição
mysql	Banco MySQL 8.4 com volume persistente (<code>mysql_data</code>), healthcheck e carga automática dos scripts da pasta <code>sql/</code> na primeira inicialização (detalhado na seção 3).
api	Aplicação GastroHub, construída a partir do <code>Dockerfile</code> , exposta na porta <code>8080</code> (configurável via variável <code>API_PORT</code>). Ativa o profile <code>prod</code> (<code>SPRING_PROFILES_ACTIVE</code>), que aponta a conexão para o MySQL através das variáveis de ambiente (<code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_NAME</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code>), e só inicia após o MySQL estar saudável (<code>depends_on</code> com <code>condition: service_healthy</code>).

Como subir o ambiente

```
docker compose up --build
```

O comando constrói a imagem da API, sobe o MySQL (com schema e dados de exemplo) e, em seguida, a API em `http://localhost:8080` .

8. Qualidade do código

Clean Architecture

- **Domínio independente:** as entidades de domínio não possuem anotações JPA nem dependência de framework — a persistência usa entidades JPA próprias, convertidas por métodos `fromDomain()` / `toDomain()`.
- **Inversão de dependência:** os casos de uso dependem de interfaces de gateway (`UserGateway` , `UserTypeGateway` , `RestaurantGateway` , `MenuItemGateway`); as implementações JPA ficam na infraestrutura.
- **Casos de uso pequenos e focados:** cada operação de negócio é um caso de uso próprio (ex.: `CreateRestaurantUseCase` , `DeleteUserTypeUseCase`), registrado como bean nas classes de configuração da infraestrutura.

Princípios SOLID Aplicados

- **S (Single Responsibility):** cada caso de uso executa uma única operação; controllers tratam apenas o fluxo HTTP; mappers apenas convertem DTOs; validações reutilizáveis são isoladas (ex.: `RestaurantDuplicationValidator`).
- **O (Open/Closed):** novas exceções de negócio podem ser adicionadas aos handlers globais sem alterar os existentes; novos casos de uso não modificam os atuais.
- **L (Liskov Substitution):** qualquer implementação de um gateway pode substituir a implementação JPA (por exemplo, um mock nos testes) sem que os casos de uso precisem de ajustes.
- **I (Interface Segregation):** cada módulo possui seu próprio gateway com apenas os métodos que utiliza, em vez de uma interface única de persistência.
- **D (Dependency Inversion):** a camada de aplicação depende de abstrações (gateways), nunca de repositórios JPA concretos.

Outras Boas Práticas

- **Entidades autovalidadas:** factory methods `create()` / `restore()` impedem a construção de objetos em estado inválido.
- **DTOs imutáveis:** requests e responses implementados como Java `record` , separados por operação.
- **Validação na entrada:** `@Valid` + Bean Validation (`@NotBlank` , `@NotNull` , `@Email` , `@DecimalMin`) nos DTOs de request.
- **Senha nunca exposta:** o campo `password` não existe no DTO de response de usuário.
- **Erros padronizados:** corpo de erro consistente em toda a API (timestamp, status, error, message).
- **Fetch LAZY:** a associação usuário → tipo de usuário usa `FetchType.LAZY` , evitando consultas desnecessárias

(problema N+1).

- **Documentação viva:** anotações OpenAPI mantidas junto aos controllers.

9. Testes automatizados

Foram implementados **163 testes automatizados** (38 classes de teste) com **JUnit 5 + Mockito + MockMvc**, distribuídos entre testes de unidade (domínio, casos de uso, gateway e mappers) e testes de integração (controllers com MockMvc e banco H2).

Distribuição dos testes

Camada / Classe de Teste	Tipo / Descrição
<code>domain/entity</code> — <code>UserTypeTest</code> , <code>UserTest</code> , <code>RestaurantTest</code> , <code>MenuItemTest</code>	Unitário: valida as regras das entidades de domínio (campos obrigatórios, formato de e-mail, preço maior que zero, factory methods <code>create</code> / <code>restore</code>) — 43 testes.
<code>application/usecase/usertype/*</code> (5 classes)	Unitário: casos de uso de tipos de usuário com gateways mockados (criação, duplicidade, exclusão, não encontrado) — 14 testes.
<code>application/usecase/user/*</code> (7 classes)	Unitário: casos de uso de usuários (criação e atualização com e-mail duplicado, busca por ID, e-mail e nome, exclusão de usuário inexistente) — 13 testes.
<code>application/usecase/restaurant/*</code> (6 classes)	Unitário: casos de uso de restaurantes (dono inexistente, dono CLIENT, duplicidade, atualização, exclusão) — 18 testes.
<code>application/usecase/menuitem/*</code> (6 classes)	Unitário: casos de uso de itens do cardápio (restaurante inexistente, criação, busca, atualização, exclusão) — 13 testes.
<code>infrastructure/persistence/gateway</code> — <code>MenuItemGatewayImplTest</code>	Persistência: gateway JPA de itens do cardápio contra o banco de teste (save, busca por ID e por restaurante, atualização, exclusão) — 8 testes.
<code>presentation/mapper/*</code> (4 classes)	Unitário: conversão domínio ⇌ DTO dos quatro módulos — 16 testes.
<code>RestaurantControllerTest</code> , <code>MenuItemControllerTest</code> , <code>UserControllerTest</code>	Integração MVC: endpoints HTTP com MockMvc, cobrindo status codes e formato das respostas — 30 testes.
<code>UserTypeControllerIntegrationTest</code>	Integração completa (<code>@SpringBootTest</code> + <code>@AutoConfigureMockMvc</code> + H2): fluxo

Camada / Classe de Teste	Tipo / Descrição
	Controller → UseCase → Gateway → JPA → H2 — 7 testes.
GastrohubApiApplicationTests	Contexto: verifica se o contexto Spring sobe corretamente — 1 teste.

Cenários cobertos pelos testes

- Criar tipo de usuário com sucesso e com nome duplicado (409)
- Excluir tipo de usuário em uso (409) e inexistente (404)
- Atualizar e buscar tipo de usuário por ID
- Criar e atualizar usuário com e-mail já cadastrado (409)
- Buscar usuário por ID, por e-mail e por nome; excluir usuário inexistente (404)
- Criar restaurante com sucesso
- Criar restaurante com dono inexistente (404) e dono da categoria CLIENT (400)
- Criar restaurante duplicado — mesmo nome, dono, endereço e cozinha (409)
- Listar, buscar por nome, atualizar e excluir restaurantes
- Criar item do cardápio com restaurante inexistente (404)
- CRUD completo de itens do cardápio e listagem por restaurante
- Validações das entidades de domínio (campos obrigatórios, preço maior que zero)
- Mapeamento correto entre entidades de domínio e DTOs

Executar os testes

```
./mvnw test
```

Resultado esperado:

```
Tests run: 163, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Os testes de integração utilizam banco **H2 in-memory** com perfil dedicado, configurado em `src/test/resources/application-test.properties` (`ddl-auto=create-drop` , modo de compatibilidade MySQL). Não é necessário nenhum software adicional para executar os testes.

10. Collection Postman

A collection do Postman está disponível no repositório, em `docs/gastrohub.postman_collection.json`, com as requisições prontas para todos os **24 endpoints** da API, organizadas em quatro pastas: **UserType**, **User**, **Restaurant** e **Menuitem**.

Como usar

1. Abra o Postman e clique em **Import**.
2. Selecione o arquivo `docs/gastrohub.postman_collection.json` do repositório.
3. Com a aplicação rodando em `http://localhost:8080`, execute as requisições de cada pasta (os corpos de exemplo já vêm preenchidos).

Alternativamente, todos os endpoints também podem ser testados pelo **Swagger UI** (seção 6), sem instalação de nenhuma ferramenta.

11. Repositório do Código

URL do Repositório

<https://github.com/GastroHubDev/gastrohub-api>

Conteúdo do Repositório

- `src/main` : código-fonte completo (domain, application, infrastructure, presentation)
- `src/test` : 163 testes automatizados
- `pom.xml` : configuração Maven e dependências
- `mvnw` / `mvnw.cmd` : Maven Wrapper (dispensa instalação do Maven)
- `Dockerfile` e `docker-compose.yml` : containerização da API e do banco MySQL
- `sql/` : scripts de schema e seed carregados pelo container MySQL
- `docs/` : relatório técnico e collection Postman
- `README.md` : documentação do projeto

Estrutura de Arquivos Principal

```
.
├─ pom.xml
├─ mvnw / mvnw.cmd
├─ README.md
├─ Dockerfile
├─ docker-compose.yml
├─ sql/
│   ├─ 01-schema.sql          (DDL das tabelas – MySQL)
│   └─ 02-seed.sql            (dados de exemplo)
├─ docs/
│   └─ gastrohub.postman_collection.json
├─ src/
│   ├─ main/
│   │   ├─ java/br/com/fiap/gastrohubapi/
│   │   │   └─ GastrohubApiApplication.java
│   │   │   └─ domain/
│   │   │       ├─ entity/          (User, UserType, Restaurant, MenuItem)
│   │   │       ├─ enums/           (BaseCategory, KitchenType)
│   │   │       └─ exception/       (10 exceções de negócio)
│   │   │   └─ application/
│   │   │       ├─ gateway/         (4 contratos de persistência)
│   │   │       └─ usecase/         (user, usertype, restaurant, menuItem)
│   │   │   └─ infrastructure/
│   │   │       ├─ config/          (registro dos use cases como beans)
│   │   │       └─ persistence/     (entity, repository, gateway)
│   │   │   └─ presentation/
│   │   │       ├─ controller/      (4 REST controllers)
│   │   │       ├─ dto/             (request / response records)
│   │   │       └─ mapper/
│   │   │           └─ exceptionhandler/
│   │   └─ resources/
│   │       ├─ application.properties (padrão: H2 local)
│   │       └─ application-prod.properties (profile prod: MySQL)
│   └─ test/
│       ├─ java/br/com/fiap/gastrohubapi/ (38 classes de teste)
│       └─ resources/
│           └─ application-test.properties
```

12. Guia passo a passo para testar o projeto

Este guia reúne tudo que é necessário para clonar, subir e testar o projeto do zero pelo Swagger UI.

Pré-requisitos: para executar com containers, apenas **Docker** (com o plugin Compose). Para executar localmente sem Docker, apenas **Java 21** (JDK) — o projeto usa o Maven Wrapper, portanto não é necessário instalar o Maven.

Etapa 1: Clonar o repositório

Abra o terminal (Prompt de Comando, PowerShell ou Terminal do Mac/Linux) e execute:

```
git clone https://github.com/GastroHubDev/gastrohub-api.git
cd gastrohub-api
```

Etapa 2: Subir a aplicação

Opção A: Com Docker (API + MySQL)

Na pasta do projeto, execute:

```
docker compose up --build
```

O Compose constrói a imagem da API e sobe o container MySQL 8.4 com o schema e os dados de exemplo carregados automaticamente (seção 7).

Opção B: Execução local (H2 in-memory)

Sem Docker, execute com o Maven Wrapper:

```
# Linux / macOS
./mvnw spring-boot:run

# Windows
.\mvnw.cmd spring-boot:run
```

Nesta opção a aplicação usa o banco H2 in-memory, criado automaticamente pelo Hibernate — nenhuma configuração de banco é necessária.

Em ambas as opções, a aplicação iniciará em `http://localhost:8080`.

Etapa 3: Testar pelo Swagger UI

3.1: Acessar o Swagger

```
http://localhost:8080/swagger-ui.html
```

A página exibirá todos os endpoints documentados da API, agrupados pelas tags **User Types**, **Users**, **Restaurants** e **Menu Items**.

3.2: Criar um tipo de usuário

1. Clique em **POST /user-types**
2. Clique no botão "Try it out"
3. Substitua o corpo de exemplo pelo JSON abaixo e clique em "Execute":

```
{
  "name": "Dono de Restaurante",
  "baseCategory": "OWNER"
}
```

A resposta esperada é **201 Created**. Copie o `id` (UUID) retornado para o próximo passo.

3.3: Criar um usuário dono de restaurante

Em **POST /users**, informe o tipo criado no passo anterior:

```
{
  "name": "Maria Oliveira",
  "email": "maria.oliveira@email.com",
  "password": "senha123",
  "userId": "<UUID copiado no passo 3.2>"
}
```

A resposta será **201 Created** com o `id` (UUID) do usuário. Copie esse UUID para o próximo passo.

3.4: Criar um restaurante

Em **POST /restaurants**, use o UUID do usuário como `restaurantOwnerId`:

```
{
  "name": "Cantina da Nonna",
  "address": "Av. Paulista, 1000 - São Paulo/SP",
  "kitchenType": "ITALIAN",
  "openingHours": "Seg a Dom, das 11h às 23h",
  "restaurantOwnerId": "<UUID copiado no passo 3.3>"
}
```

A resposta será **201 Created**. Copie o **UUID** do restaurante.

3.5: Criar um item do cardápio

Em **POST /menu-items**, use o **UUID** do restaurante:

```
{
  "name": "Lasanha à Bolonhesa",
  "description": "Massa fresca com molho bolonhesa e queijo gratinado",
  "price": 59.90,
  "onlyInRestaurant": false,
  "photoPath": "/images/pratos/lasanha-bolonhesa.png",
  "restaurantId": "<UUID copiado no passo 3.4>"
}
```

3.6: Explorar os demais endpoints

- **GET /restaurants**: lista todos os restaurantes
- **GET /menu-items/restaurant/{restaurantId}**: lista o cardápio do restaurante
- **GET /users/search?name=Maria**: busca usuários pelo nome
- **PUT** e **DELETE** nos módulos para atualizar/excluir registros
- Teste os cenários de erro: criar restaurante com dono **CLIENT** (400), excluir tipo de usuário em uso (409), buscar ID inexistente (404)

3.7: Inspecionar o banco (opcional)

Na execução local (Opção B), o console do H2 está habilitado em:

```
http://localhost:8080/h2-console

JDBC URL: jdbc:h2:mem:gastrohub
User:     sa
Senha:    (em branco)
```

Etapa 4: Executar os testes automatizados

```
# Linux / macOS
./mvnw test

# Windows
.\mvnw.cmd test
```

Observação: na execução local com H2 (Opção B), o banco é in-memory e os dados são perdidos ao reiniciar a aplicação — repita as etapas 3.2 a 3.5 após cada reinício.

13. Observações e Limitações conhecidas

Esta seção documenta limitações identificadas durante o desenvolvimento e os testes da Fase 2, que estão fora do escopo atual ou serão tratadas antes da entrega final.

Banco H2 na execução local

O banco definitivo de produção é o **MySQL em container** (seções 3 e 7): no ambiente Docker, o Compose ativa o profile `prod` (`application-prod.properties`), que parametriza a conexão pelas variáveis de ambiente `DB_HOST` , `DB_PORT` , `DB_NAME` , `DB_USER` e `DB_PASSWORD` . Na execução local de desenvolvimento e nos testes automatizados, a aplicação utiliza **H2 in-memory**, cujos dados são perdidos a cada reinício.

Ausência de autenticação

Nesta versão não há camada de segurança: todos os endpoints são públicos.

Senha armazenada sem hash

A senha do usuário é validada como obrigatória e nunca retornada nas respostas, porém ainda é persistida sem algoritmo de hash. O hashing poderá ser incorporado em evolução futura, junto com uma camada de autenticação.

Ausência de paginação

Os endpoints de listagem (`GET /users` , `GET /restaurants` , `GET /menu-items` etc.) retornam todos os registros sem paginação. Em fases futuras será implementada a paginação com `Pageable` .

Relacionamentos por referência de ID

As associações restaurante → dono e item do cardápio → restaurante são mantidas por UUID (validado nos casos de uso), sem chave estrangeira física no banco. A integridade referencial é garantida em nível de aplicação.